

# Chapter 10

---

## Editing Records and Performing Queries



Once you have your databases and tables defined, the next step is to begin using them, by populating them with records and performing queries on the data stored inside them. To do this, you need to master the Data Manipulation Languages (DML) component of SQL. That's why this chapter discusses the SQL commands to add, edit, and delete records to a table, and then perform different types of queries on that data to retrieve a result set of records that satisfy the query.

## How to...

- Use the `INSERT` command to add records to a table
- Edit and delete records with the `UPDATE` and `DELETE` commands
- Retrieve table records with the `SELECT` command
- Use the `WHERE` clause to restrict the scope of your actions to a subset of records
- Use functions and operators to manipulate and compare field values
- Sort result sets and eliminate duplicate values
- Set a limit on the number of records in a result set
- Group records in a result set using common attributes (and then perform operations on these groups)
- Join tables together using foreign keys
- Use the results of one query inside another

## Inserting Records

Once you've created a table, it's time to begin entering data into it-and the SQL command to accomplish this is the `INSERT` command. The syntax of the `INSERT` command is illustrated in the following example:

```
mysql> INSERT INTO movies (mtitle, myear) VALUES ('Rear Window', 1954);  
Query OK, 1 row affected (0.06 sec)
```

The `INSERT` command is followed by the optional keyword `INTO`, a table name, and a field list, in parentheses, which indicates which fields the values are to be inserted into. A `VALUES` clause completes the command, by specifying the values to be inserted into the previously named fields.

You can also use an abbreviated form of the `INSERT` command, in which the field list is left unspecified. The following example, which is equivalent to the previous one illustrates the following:

```
mysql> INSERT INTO movies VALUES (NULL, 'Rear Window', 1954);
Query OK, 1 row affected (0.06 sec)
```

When using this shorter format, the order in which values are inserted must correspond to the sequence of fields in the table (you can determine the field order with a quick call to the `DESCRIBE` command, described in Chapter 9).

Normally, the first version of the `INSERT` command is preferable, because it offers you the flexibility of inserting values in any order you please and protects you from structural changes in the table. Because of this, the following statements are equivalent:

```
mysql> INSERT INTO movies (mtitle, myear) VALUES ('Rear Window', 1954);
Query OK, 1 row affected (0.06 sec)
```

```
mysql> INSERT INTO movies (myear, mtitle) VALUES (1954, 'Rear Window');
Query OK, 1 row affected (0.06 sec)
```

In MySQL, you can insert multiple records into a table at once, by using multiple `VALUES ()` clauses within the same `INSERT` statement. To see how this works, try running the following command:

```
mysql> INSERT INTO movies (mtitle, myear) VALUES ('Rear Window', 1954),
('To Catch A Thief', 1955), ('The Maltese Falcon', 1941);
Query OK, 3 rows affected (0.12 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

Fields that are not specified in the `INSERT` command will either be set to `NULL` or to their default values, depending on how they have been defined. MySQL comes with built-in intelligence to automatically deal with conflicts

between the values entered into a field and the field's data type, or with missing values, and so can automatically perform the following operations:

- For `AUTO_INCREMENT` fields, entering a `NULL` value automatically increments the previously generated field value by 1.
- For the first `TIMESTAMP` field in a table, entering a `NULL` value automatically inserts the current date and time.
- For `UNIQUE` or `PRIMARY KEY` fields, entering a value that already exists causes MySQL to generate an error.

### TIP

*When inserting string and some date values into a table, enclose them in quotation marks, so that MySQL doesn't confuse them with variable or field names. Quotation marks within the values themselves can be "escaped" by preceding them with the backslash ( \ ) symbol.*

Now that you know how to insert records, try inserting some sample records for the three tables created in the previous section, using the sample data in Chapter 8 as a reference. You can start with these samples:

```
mysql> INSERT INTO movies VALUES (1,'Rear Window',1954);
Query OK, 1 row affected (0.06 sec)
mysql> INSERT INTO persons VALUES (1,'Alfred Hitchcock','M','1899-08-13');
Query OK, 1 row affected (0.06 sec)
mysql> INSERT INTO roles VALUES (1,1,'D'), (1,3,'A');
Query OK, 2 rows affected (0.06 sec)
```

## Editing and Deleting Records

Just as you `INSERT` records into a table, you can also `DELETE` records with the `DELETE` command, which is illustrated in the following:

```
mysql> DELETE FROM movies;
Query OK, 0 rows affected (0.06 sec)
```

The previous command would delete all the records from the `movies` table.

You can select a specific subset of rows to be deleted by adding the `WHERE` clause to the `DELETE` statement. The following example would only delete records for those persons born after 1960:

```
mysql> DELETE FROM movies WHERE myear > 1960;
Query OK, 1 row affected (0.05 sec)
```

**CAUTION**

*It is not possible to reverse a DELETE operation in MySQL (unless you're in the middle of a InnoDB transaction which hasn't yet been committed). Therefore, be extremely careful when using DELETE commands, both with and without WHERE clauses—a small mistake and the contents of your entire table will be lost for good.*

**TIP**

*To delete all the records in a table, consider using the TRUNCATE TABLE command, described in Chapter 9.*

Data in a database usually changes over time, which is why SQL includes an UPDATE command designed to change existing values in a table. As with the DELETE command described previously, you can use the UPDATE command to change all the values in a particular column, or change only those values matching a particular condition.

To illustrate how this works, consider the following example, which changes the value of the field 'The Maltese Falcon' to 'Maltese Falcon, The'.

```
mysql> UPDATE movies SET mtitle = 'Maltese Falcon, The' ␣
WHERE mtitle = 'The Maltese Falcon';
Query OK, 1 row affected (0.05 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

10

You can update multiple fields at once, simply by using multiple SET clauses. The following example illustrates, by updating record #7 with a new movie title and year:

```
mysql> UPDATE movies SET mtitle = 'Vertigo', myear = 1958 WHERE mid = 7;
Query OK, 1 row affected (0.06 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Thus, the SET clause specifies the field name, as well as the new value for the field. The WHERE clause is used to identify which rows of the table to change. In the absence of this clause, all the rows of the table are updated with the new value. Try this out by entering the following command, which updates the psex field in the persons table:

```
mysql> UPDATE persons SET psex = 'M';
Query OK, 1 row affected (0.06 sec)
Rows matched: 6  Changed: 1  Warnings: 0
```

Did you  
know?

## Multitasking with MySQL

Newer versions of MySQL enable you to update and delete records in multiple tables simultaneously with a single query.

If you look at the table now, you will see that all the records in the table sport the value M for their psex field. Correct it by again using an UPDATE command with a WHERE clause:

```
mysql> UPDATE persons SET psex = 'F' WHERE pname = 'Grace Kelly';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1   Changed: 1   Warnings: 0
```

### CAUTION

*Forgetting the WHERE clause in an UPDATE command is a common newbie mistake, and it can lead to widespread data corruption. Always use a WHERE clause to restrict the effect of the UPDATE to relevant fields only.*

## Performing Queries

Just as you can add records to a table with the INSERT command, you can retrieve them with the SELECT command. The SELECT command is one of the most versatile and useful commands in SQL. It offers tremendous flexibility in extracting specific subsets of data from a table.

In its most basic form, the SELECT statement can be used to evaluate expressions and functions, or as a “catch-all” query that returns all the records in a specific table. Here is an example of using SELECT to evaluate mathematical expressions:

```
mysql> SELECT 75 / 15, 61 + (3 * 3);
+-----+-----+
| 75 / 15 | 61 + (3 * 3) |
+-----+-----+
|      5.00 |              70 |
+-----+-----+
1 row in set (0.05 sec)
```

And here is an example of using `SELECT` to retrieve all the records in a table:

```
mysql> SELECT * FROM movies;
+-----+-----+-----+
| mid | mtitle                | myear |
+-----+-----+-----+
| 1   | Rear Window           | 1954  |
| 2   | To Catch A Thief      | 1955  |
| 3   | The Maltese Falcon     | 1941  |
| 4   | The Birds             | 1963  |
| 5   | North By Northwest     | 1959  |
| 6   | Casablanca            | 1942  |
| 7   | Anatomy Of A Murder   | 1959  |
+-----+-----+-----+
7 rows in set (0.00 sec)
```

## Retrieving Specific Columns

The asterisk (\*) in the previous example indicates that you'd like the output of `SELECT` to contain all the columns present in the table. If, instead, you'd prefer to see one or two specific columns only in the result set, you can specify the column name(s) in the `SELECT` statement, like this:

```
mysql> SELECT mtitle FROM movies;
+-----+
| mtitle                |
+-----+
| Rear Window           |
| To Catch A Thief      |
| The Maltese Falcon     |
| The Birds             |
| North By Northwest     |
| Casablanca            |
| Anatomy Of A Murder   |
+-----+
7 rows in set (0.00 sec)
```

## Filtering Records with a WHERE Clause

You can also restrict which records appear in the result set, by adding a `WHERE` clause to your `SELECT` statement. This `WHERE` clause lets you define specific criteria used to filter records from the result set. Records that do not meet the specified criteria will not appear in the result set.

## How to ...

**Refer to Fields Clearly**

When dealing with multiple tables, a good idea is to prefix the field name with the table name so it is immediately clear which table each field belongs to. This is of particular importance when joining tables to each other through common fields. For example, the query `SELECT a.name, b.dob from a,b where a.id = b.id` makes it clear that the name field belongs to table a and the dob field belongs to table b. See the section entitled “Joining Tables” to see many more examples of this in practice.

For example, suppose you want to find out which year *Casablanca* was released:

```
mysql> SELECT myyear FROM movies WHERE mtitle = 'Casablanca';
+-----+
| myear |
+-----+
|  1942 |
+-----+
1 row in set (0.11 sec)
```

**Using Operators**

The = symbol previously used is an equality operator, used to test whether the left side of the expression is equal to the right side. MySQL comes with numerous such operators that can be used in the WHERE clause for comparisons and calculations. Table 10-1 lists the important operators in MySQL, by category.

Here is an example of using a comparison operator in the WHERE clause, to list all movies released after 1950:

```
mysql> SELECT myyear, mtitle FROM movies WHERE myear > 1950;
+-----+-----+
| myear | mtitle |
+-----+-----+
```



```
| 1954 | Rear Window |
| 1955 | To Catch A Thief |
| 1963 | The Birds |
| 1959 | North By Northwest |
| 1959 | Anatomy Of A Murder |
+-----+-----+
5 rows in set (0.00 sec)
```

| Operator                    | What It Does               |
|-----------------------------|----------------------------|
| <i>Arithmetic operators</i> |                            |
| +                           | Addition                   |
| -                           | Subtraction                |
| *                           | Multiplication             |
| /                           | Division; returns quotient |
| %                           | Division; returns modulus  |
| <i>Comparison operators</i> |                            |
| =                           | Equal to                   |
| <> aka !=                   | Not equal to               |
| <=>                         | NULL-safe equal to         |
| <                           | Less than                  |
| <=                          | Less than or equal to      |
| >                           | Greater than               |
| >=                          | Greater than or equal to   |
| BETWEEN                     | Exists in specified range  |
| IN                          | Exists in specified set    |
| IS NULL                     | Is a NULL value            |
| IS NOT NULL                 | Is not a NULL value        |
| LIKE                        | Wildcard match             |
| REGEXP aka RLIKE            | Regular expression match   |
| <i>Logical operators</i>    |                            |
| NOT aka !                   | Logical NOT                |
| AND aka &&                  | Logical AND                |
| OR aka                      | Logical OR                 |
| XOR                         | Exclusive OR               |

**TABLE 10-1** MySQL Operators

You can combine multiple conditions by using the AND or OR logical operators. This next example lists all movies released between 1955 and 1965:

```
mysql> SELECT mtitle FROM movies WHERE myear >= 1955 AND myear <= 1965;
+-----+
| mtitle                |
+-----+
| To Catch A Thief     |
| The Birds             |
| North By Northwest    |
| Anatomy Of A Murder   |
+-----+
4 rows in set (0.06 sec)
```

Another way to perform this comparison is with the BETWEEN operator:

```
mysql> SELECT mtitle FROM movies WHERE myear BETWEEN 1955 AND 1965;
+-----+
| mtitle                |
+-----+
| To Catch A Thief     |
| The Birds             |
| North By Northwest    |
| Anatomy Of A Murder   |
+-----+
4 rows in set (0.06 sec)
```

The LIKE operator can be used to perform queries using wildcards, and comes in handy when you're not sure what you're looking for. Two types of wildcards are allowed when using the LIKE operator: the % wildcard, which is used to signify zero or more occurrences of a character, and the \_ wildcard, which is used to signify exactly one occurrence of a character.

This next example uses the LIKE operator with the logical OR operator to list all movie titles containing the letters *m* or *n*:

```
mysql> SELECT mtitle FROM movies WHERE mtitle LIKE '%m%'
OR mtitle LIKE '%n%';
+-----+
| mtitle                |
+-----+
| Rear Window           |
```

```
| The Maltese Falcon |
| North By Northwest |
| Casablanca        |
| Anatomy Of A Murder |
+-----+
5 rows in set (0.06 sec)
```

## Sorting Records and Eliminating Duplicates

If you'd like to see the data from your table ordered by a specific field, SQL offers the `ORDER BY` clause. This clause enables you to specify both the column name and the direction in which you would like to see data (ASCending or DESCending).

Here is an example of sorting the `persons` table by name, in ascending order:

```
mysql> SELECT * FROM persons ORDER BY pname ASC;
+-----+-----+-----+-----+
| pid | pname                | psex | pdob          |
+-----+-----+-----+-----+
| 1   | Alfred Hitchcock     | M    | 1899-08-13    |
| 2   | Cary Grant           | M    | 1904-01-18    |
| 3   | Grace Kelly          | F    | 1929-11-12    |
| 4   | Humphrey Bogart      | M    | 1899-12-25    |
| 6   | James Stewart        | M    | 1908-05-20    |
| 5   | Sydney Greenstreet   | M    | 1879-12-27    |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

And here is the same table sorted by date of birth, in descending order:

```
mysql> SELECT * FROM persons ORDER BY pdob DESC;
+-----+-----+-----+-----+
| pid | pname                | psex | pdob          |
+-----+-----+-----+-----+
| 3   | Grace Kelly          | F    | 1929-11-12    |
| 6   | James Stewart        | M    | 1908-05-20    |
| 2   | Cary Grant           | M    | 1904-01-18    |
| 4   | Humphrey Bogart      | M    | 1899-12-25    |
| 1   | Alfred Hitchcock     | M    | 1899-08-13    |
| 5   | Sydney Greenstreet   | M    | 1879-12-27    |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Did you  
know?

## Need for Speed

MySQL 4.0 includes a query cache, which can substantially improve performance by caching the results of common queries and returning this cached data to the caller without having to reexecute the query each time.

To eliminate duplicate records in a table, add the `DISTINCT` keyword. Consider the following example, which illustrates the use of this keyword by printing a list of all the unique year values in the `movies` table:

```
mysql> SELECT DISTINCT myear FROM movies;
```

```
+-----+
| myear |
```

```
+-----+
```

```
| 1954 |
| 1955 |
| 1941 |
| 1963 |
| 1959 |
| 1942 |
```

```
+-----+
```

```
6 rows in set (0.06 sec)
```

## Limiting Results

You can limit the number of records returned by MySQL with the `LIMIT` clause, as illustrated in the following:

```
mysql> SELECT mtitle FROM movies LIMIT 0,4;
```

```
+-----+
| mtitle |
```

```
+-----+
```

```
| Rear Window |
| To Catch A Thief |
| The Maltese Falcon |
| The Birds |
```

```
+-----+
```

```
4 rows in set (0.00 sec)
```

You can even combine the `ORDER BY` and `LIMIT` clauses to return a sorted list restricted to a certain number of values. The following example illustrates, by listing the three oldest people (as per their birth dates) in the `persons` table:

```
mysql> SELECT pname FROM persons ORDER BY pdob LIMIT 0,3;
+-----+
| pname                |
+-----+
| Sydney Greenstreet   |
| Alfred Hitchcock     |
| Humphrey Bogart      |
+-----+
3 rows in set (0.00 sec)
```

## Using Built-In Functions

MySQL comes with over 100 built-in functions to help you perform calculations and process the records in a result set. These functions can be used in a `SELECT` statement, either to manipulate field values or in the `WHERE` clause. The following example illustrates, by using MySQL's `COUNT()` function to return the total number of records in the `movies` table:

```
mysql> SELECT COUNT(*) FROM movies;
+-----+
| COUNT(*) |
+-----+
|         7 |
+-----+
1 row in set (0.00 sec)
```

You can calculate string length with the `LENGTH()` function, as in the following:

```
mysql> SELECT pname, LENGTH(pname) FROM persons;
+-----+-----+
| pname                | LENGTH(pname) |
+-----+-----+
| Alfred Hitchcock     |             16 |
| Cary Grant           |             10 |
| Grace Kelly          |             11 |
| Humphrey Bogart      |             15 |
| Sydney Greenstreet   |             18 |
| James Stewart        |             13 |
+-----+-----+
6 rows in set (0.00 sec)
```

You can use the `DATE()` function to format date and time values into a human-readable form, as illustrated in the following:

```
mysql> SELECT pname, DATE_FORMAT(pdob, '%W %d %M %Y') FROM persons;
```

| pname              | DATE_FORMAT(pdob, '%W %d %M %Y') |
|--------------------|----------------------------------|
| Alfred Hitchcock   | Sunday 13 August 1899            |
| Cary Grant         | Monday 18 January 1904           |
| Grace Kelly        | Tuesday 12 November 1929         |
| Humphrey Bogart    | Monday 25 December 1899          |
| Sydney Greenstreet | Saturday 27 December 1879        |
| James Stewart      | Wednesday 20 May 1908            |

```
6 rows in set (0.00 sec)
```

You can even use functions in the `WHERE` clause of a `SELECT` statement. The following example illustrates, by listing all those people who would be more than 100 years old today if they were still alive:

```
mysql> SELECT pname FROM persons WHERE YEAR(NOW()) - YEAR(pdob) > 100;
```

| pname              |
|--------------------|
| Alfred Hitchcock   |
| Humphrey Bogart    |
| Sydney Greenstreet |

```
3 rows in set (0.06 sec)
```

## Grouping Records

You can group records on the basis of a specific field with MySQL's `GROUP BY` clause. Each group created in this manner is treated as a single row, even though it internally contains multiple records. Consider the following example, which groups the records in the `persons` table on the basis of their sex:

```
mysql> SELECT * FROM persons GROUP BY psex;
```

| pid | pname | psex | pdob |
|-----|-------|------|------|
|-----|-------|------|------|

```
+-----+-----+-----+-----+
| 1 | Alfred Hitchcock | M | 1899-08-13 |
| 3 | Grace Kelly      | F | 1929-11-12 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

A number of specialized functions are available when grouping records in this manner. The most commonly used one in this context is the `COUNT()` function, which you saw earlier. In the context of a `GROUP BY` clause, this function can be used to count the number of records in each group. The following example illustrates by counting the number of males and females in the `persons` table:

```
mysql> SELECT psex, COUNT(psex) FROM persons GROUP BY psex;
+-----+-----+
| psex | COUNT(psex) |
+-----+-----+
| M    | 5           |
| F    | 1           |
+-----+-----+
2 rows in set (0.00 sec)
```

Here's another example, this one returning the number of persons linked to each movie in the `roles` table:

```
mysql> SELECT mid, COUNT(pid) FROM roles GROUP BY mid;
+-----+-----+
| mid | COUNT(pid) |
+-----+-----+
| 1   | 3           |
| 2   | 3           |
| 3   | 2           |
| 4   | 1           |
| 5   | 2           |
| 6   | 1           |
+-----+-----+
6 rows in set (0.06 sec)
```

You can further filter the groups by adding a `HAVING` clause to the `GROUP BY` clause. This `HAVING` clause works much like a regular `WHERE` clause, letting you further filter the grouped data by a specific condition. The following example

revises the previous one to only return those movies having two or more persons linked to them:

```
mysql> SELECT mid, COUNT(pid) FROM roles GROUP BY mid
HAVING COUNT(pid) >= 2;
```

```
+-----+-----+
| mid | COUNT(pid) |
+-----+-----+
| 1 | 3 |
| 2 | 3 |
| 3 | 2 |
| 5 | 2 |
+-----+-----+
4 rows in set (0.00 sec)
```

## Joining Tables

So far, all the queries you've seen have been concentrated on a single table. But SQL also enables you to query two or more tables at a time, and to display a combined result set. This is technically referred to as a *join*, because it involves “joining” different tables at specific points to create new views of the data. MySQL has supported joins well right from its inception, and today boasts support for standard SQL2-compliant join syntax, which makes it possible to combine table records in a variety of sophisticated ways.

When using a join, the recommendation is that you prefix each field name with the name of the table it belongs to. For example, you would use `movies.mid` to refer to the field named `mid` in the table `movies`, and `roles.pid` to refer to the `pid` field in the `roles` table.

Did you  
know?

## Playing the Numbers

In addition to the `COUNT()` function, MySQL also offers the `MIN()` and `MAX()` functions to retrieve the minimum and maximum of a group, the `AVG()` function to return the average of a group of values, and the `SUM()` function to return the total of a group of values.



## Inner Joins

Here's an example of a simple join:

```
mysql> SELECT * FROM movies, roles WHERE movies.mid = roles.mid;
```

| mid | mtitle              | myear | mid | pid | role |
|-----|---------------------|-------|-----|-----|------|
| 1   | Rear Window         | 1954  | 1   | 1   | D    |
| 1   | Rear Window         | 1954  | 1   | 3   | A    |
| 1   | Rear Window         | 1954  | 1   | 6   | A    |
| 2   | To Catch A Thief    | 1955  | 2   | 1   | D    |
| 2   | To Catch A Thief    | 1955  | 2   | 2   | A    |
| 2   | To Catch A Thief    | 1955  | 2   | 3   | A    |
| 3   | Maltese Falcon, The | 1941  | 3   | 4   | A    |
| 3   | Maltese Falcon, The | 1941  | 3   | 5   | A    |
| 4   | The Birds           | 1963  | 4   | 1   | D    |
| 5   | North By Northwest  | 1959  | 5   | 1   | D    |
| 5   | North By Northwest  | 1959  | 5   | 2   | A    |
| 6   | Casablanca          | 1942  | 6   | 4   | A    |

12 rows in set (0.00 sec)

10

In this case, the `movies` and `roles` tables have been joined together through the common field `mid`. Such a join is referred to as an *inner join*, because its result set contains only those records that match in all the tables in the join. Records that do not match are excluded from the final result set.

Did you  
know?

## The Inner Circle

Inner joins are the most common type of join you'll see in this book (and in your PHP-MySQL development). Specifically, the previous join is known as an *equijoin*, because it attempts to equate records in one table with records in another. You can also create inner joins using inequalities between fields in different tables. In this case, the final result set will only include those rows from the joined tables that have matches in the specified fields.

You can also use `INNER JOIN` syntax to make things clearer. This next example, which is equivalent to the previous one, illustrates.

```
mysql> SELECT * FROM movies INNER JOIN roles USING (mid);
```

| mid | mtitle              | myear | mid | pid | role |
|-----|---------------------|-------|-----|-----|------|
| 1   | Rear Window         | 1954  | 1   | 1   | D    |
| 1   | Rear Window         | 1954  | 1   | 3   | A    |
| 1   | Rear Window         | 1954  | 1   | 6   | A    |
| 2   | To Catch A Thief    | 1955  | 2   | 1   | D    |
| 2   | To Catch A Thief    | 1955  | 2   | 2   | A    |
| 2   | To Catch A Thief    | 1955  | 2   | 3   | A    |
| 3   | Maltese Falcon, The | 1941  | 3   | 4   | A    |
| 3   | Maltese Falcon, The | 1941  | 3   | 5   | A    |
| 4   | The Birds           | 1963  | 4   | 1   | D    |
| 5   | North By Northwest  | 1959  | 5   | 1   | D    |
| 5   | North By Northwest  | 1959  | 5   | 2   | A    |
| 6   | Casablanca          | 1942  | 6   | 4   | A    |

```
12 rows in set (0.00 sec)
```

You can join as many tables as you like in this manner. This next example adds the `persons` table to the previous join, and it also selects the rows and columns to be displayed in the output of the join by specifying them in the `SELECT` statement:

```
mysql> SELECT movies.mtitle, persons.pname, roles.role
FROM movies, persons, roles WHERE movies.mid = roles.mid
AND persons.pid = roles.pid;
```

| mtitle              | pname              | role |
|---------------------|--------------------|------|
| Rear Window         | Alfred Hitchcock   | D    |
| Rear Window         | Grace Kelly        | A    |
| Rear Window         | James Stewart      | A    |
| To Catch A Thief    | Alfred Hitchcock   | D    |
| To Catch A Thief    | Cary Grant         | A    |
| To Catch A Thief    | Grace Kelly        | A    |
| Maltese Falcon, The | Humphrey Bogart    | A    |
| Maltese Falcon, The | Sydney Greenstreet | A    |

|                           |                    |  |                  |  |   |  |
|---------------------------|--------------------|--|------------------|--|---|--|
|                           | The Birds          |  | Alfred Hitchcock |  | D |  |
|                           | North By Northwest |  | Alfred Hitchcock |  | D |  |
|                           | North By Northwest |  | Cary Grant       |  | A |  |
|                           | Casablanca         |  | Humphrey Bogart  |  | A |  |
| +-----+-----+-----+       |                    |  |                  |  |   |  |
| 12 rows in set (0.00 sec) |                    |  |                  |  |   |  |

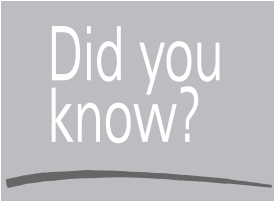
Obviously, you can add more WHERE clauses to this join to further filter the result set. For example, this next query prints a list of all those movies directed by Alfred Hitchcock:

```
mysql> SELECT movies.mtitle, persons.pname, roles.role ↵
FROM movies, persons, roles WHERE movies.mid = roles.mid ↵
AND persons.pid = roles.pid AND roles.role = 'D' ↵
AND persons.pname = 'Alfred Hitchcock';
```

|                          |                    |  |                  |  |      |  |
|--------------------------|--------------------|--|------------------|--|------|--|
|                          | mtitle             |  | pname            |  | role |  |
|                          | Rear Window        |  | Alfred Hitchcock |  | D    |  |
|                          | To Catch A Thief   |  | Alfred Hitchcock |  | D    |  |
|                          | The Birds          |  | Alfred Hitchcock |  | D    |  |
|                          | North By Northwest |  | Alfred Hitchcock |  | D    |  |
| +-----+-----+-----+      |                    |  |                  |  |      |  |
| 4 rows in set (0.06 sec) |                    |  |                  |  |      |  |

Outer Joins

MySQL also supports *outer joins*, which are asymmetrical-all records from one side of the join are included in the final result set, regardless of whether they match



Joining Up

Inner and outer joins are not the only types of joins supported in MySQL. You can also use a *cross join* to multiply the contents of both tables together; a *self join* to join a table to a new, virtual copy of itself; and a *union* to join together the results of two SELECT queries. To read more about these types of joins and view examples, look in the online MySQL manual, at <http://dev.mysql.com/doc/mysql/en/JOIN.html>.

records on the other side of the join. Consider the following example, which illustrates by using a *left outer join* to connect the `movies` table to the `roles` table:

```
mysql> SELECT * FROM movies LEFT JOIN roles
ON movies.mid = roles.mid;
```

| mid | mtitle              | myear | mid  | pid  | role |
|-----|---------------------|-------|------|------|------|
| 1   | Rear Window         | 1954  | 1    | 1    | D    |
| 1   | Rear Window         | 1954  | 1    | 3    | A    |
| 1   | Rear Window         | 1954  | 1    | 6    | A    |
| 2   | To Catch A Thief    | 1955  | 2    | 1    | D    |
| 2   | To Catch A Thief    | 1955  | 2    | 2    | A    |
| 2   | To Catch A Thief    | 1955  | 2    | 3    | A    |
| 3   | Maltese Falcon, The | 1941  | 3    | 4    | A    |
| 3   | Maltese Falcon, The | 1941  | 3    | 5    | A    |
| 4   | The Birds           | 1963  | 4    | 1    | D    |
| 5   | North By Northwest  | 1959  | 5    | 1    | D    |
| 5   | North By Northwest  | 1959  | 5    | 2    | A    |
| 6   | Casablanca          | 1942  | 6    | 4    | A    |
| 7   | Vertigo             | 1958  | NULL | NULL | NULL |

13 rows in set (0.06 sec)

As you can see, all the rows from the table on the left side of the join appear in the final result set. Those that have a corresponding value in the table on the right side as per the match condition have that value displayed; the rest have a `NULL` value displayed.

This kind of join comes in handy when you need to see which values from one table are missing in another table—all you need to do is look for the `NULL` rows. From a quick glance at the previous example, you can see that entries for all the movies in the `movies` table exist in the `roles` table, except for the movie *Vertigo*. Thus, outer joins come in handy when you're looking for corrupted, or "dirty," data in interrelated tables.

## TIP

Use the `IS NULL` operator to automatically isolate `NULL` rows in a left or right join.

Just as there is a left outer join, there also exists a *right outer join*, which works in reverse. A right outer join displays all the records from the table on the right side of the join, and then tries to match them with records from the table on the left side of the join.

## Using Subqueries

*Subqueries*, as the name suggests, are queries nested inside other queries. They make it possible to use the results of one query directly in the conditional tests or FROM clauses of other queries. Subqueries can substantially simplify the task of writing SQL-based applications, by reducing the number of application-level query statements to be executed in a given program.

Subqueries come in many shapes, sizes, and forms. The most common is a SELECT within a SELECT, such that the results of the inner SELECT serve as values for the WHERE clause of the outer SELECT. However, while this is certainly one of the most common uses of subqueries, it's not the only one. You can use subqueries in a number of other places, including within grouped result sets, with comparison and logical operators, with membership tests, in UPDATE and DELETE operations, and within a query's FROM clause.

### NOTE

*Subqueries are new to MySQL, so they are only available in MySQL 4.1 and above.*

10

To see how a subquery works, try out the following example, which prints a list of all those movie IDs starring Cary Grant:

```
mysql> SELECT mid FROM roles WHERE role = 'A'
AND pid = (SELECT pid FROM persons WHERE pname = 'Cary Grant');
+-----+
| mid |
+-----+
| 2 |
| 5 |
+-----+
2 rows in set (0.00 sec)
```

Here, the inner query is executed first, and returns the ID of the record for “Cary Grant” from the *persons* table. This ID (#1) is then substituted in the outer query's WHERE clause, and the query is executed on the *roles* table to list all those movies in which he performed.

However, this is still incomplete—the previous double query only returns a list of movie IDs, not titles. For this to be truly valuable, you need the movie titles. So, wrap the previous combination in yet another query, which takes the list of IDs generated and matches them against the `movies` table to return the corresponding titles:

```
mysql> SELECT mtitle FROM movies WHERE mid IN (
(SELECT mid FROM roles WHERE role = 'A' AND pid = (
(SELECT pid FROM persons WHERE pname = 'Cary Grant')));
+-----+
| mtitle          |
+-----+
| To Catch A Thief |
| North By Northwest |
+-----+
2 rows in set (0.06 sec)
```

Thus, a subquery makes it possible to combine two or more queries into a single statement, and to use the results of one query in the conditional clause of the other. Subqueries are usually regular `SELECT` statements, separated from their parent query by parentheses. As the previous example illustrates, you can nest subqueries to any depth, as long as the basic rules are followed.

Did you  
know?

## You Say Tom-Ah-To, I Say Tom-Ay-To...

Most of the time, subqueries can be rewritten as joins, and vice versa. For example, the queries `SELECT x FROM a WHERE y = (SELECT y FROM b WHERE condition)` and `SELECT x FROM a, b WHERE a.y = b.y AND condition` are equivalent. However, because subquery support in MySQL is still experimental, joins currently offer better performance than subqueries. Read more at <http://dev.mysql.com/doc/mysql/en/Subqueries.html>.

## Using Table and Column Aliases

For table and field names that are either too long to comfortably use or too complex to read, use the AS keyword to alias the name to a different value. The following example demonstrates, by aliasing the name of the persons table to p and the psex, pname, and pdob fields to Sex, realName, and DateOfBirth:

```
mysql> SELECT p.psex AS Sex, p.pname AS RealName, ↵
p.pdob AS DateOfBirth FROM persons AS p;
```

| Sex | RealName           | DateOfBirth |
|-----|--------------------|-------------|
| M   | Alfred Hitchcock   | 1899-08-13  |
| M   | Cary Grant         | 1904-01-18  |
| F   | Grace Kelly        | 1929-11-12  |
| M   | Humphrey Bogart    | 1899-12-25  |
| M   | Sydney Greenstreet | 1879-12-27  |
| M   | James Stewart      | 1908-05-20  |

```
6 rows in set (0.00 sec)
```

10

This also works on fields that are the result of a calculation or function operation. The following examples demonstrate this:

```
mysql> SELECT COUNT(*) AS total FROM movies;
```

```
+-----+
| total |
```

```
+-----+
|      7 |
```

```
+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> SELECT pname AS name, YEAR(NOW()) - YEAR (pdob) AS age ↵
FROM persons ORDER BY age;
```

| name             | age |
|------------------|-----|
| Grace Kelly      | 75  |
| James Stewart    | 96  |
| Cary Grant       | 100 |
| Alfred Hitchcock | 105 |

```
| Humphrey Bogart | 105 |  
| Sydney Greenstreet | 125 |  
+-----+-----+  
6 rows in set (0.05 sec)
```

**TIP**

*For many more examples of building sophisticated SELECT queries, visit Chapter 12.*

## Summary

This chapter took a big step forward in your MySQL education, showing you how to add, update, and remove data from a MySQL table, so you can begin using MySQL to store information. It also showed you how to do something with all that data once you have it safely inserted into one or more tables, by giving you a crash course in the SELECT statement and its numerous variants. The SELECT statement is one of the most versatile and useful commands in the SQL lexicon. You'll be using it frequently when you build PHP-MySQL applications.

While this chapter covered a fair bit of ground, it still barely scratched the surface of what you can do with MySQL. For more in-depth information about the topics in this chapter, you should visit the following links:

- The INSERT statement, at <http://dev.mysql.com/doc/mysql/en/INSERT.html>
- The UPDATE statement, at <http://dev.mysql.com/doc/mysql/en/UPDATE.html>
- The DELETE statement, at <http://dev.mysql.com/doc/mysql/en/DELETE.html>
- More examples of using the SELECT statement in the MySQL manual, at <http://dev.mysql.com/doc/mysql/en/SELECT.html>
- MySQL operators, at [http://dev.mysql.com/doc/mysql/en/Non-typed\\_Operators.html](http://dev.mysql.com/doc/mysql/en/Non-typed_Operators.html)
- Built-in MySQL functions, at <http://dev.mysql.com/doc/mysql/en/Functions.html>
- Date and time functions in MySQL, at <http://www.melonfire.com/community/columns/trog/article.php?id=241>



- String functions in MySQL, at <http://www.melonfire.com/community/columns/trog/article.php?id=235>
- Group manipulation functions in MySQL, at [http://dev.mysql.com/doc/mysql/en/Group\\_by\\_functions\\_and\\_modifiers.html](http://dev.mysql.com/doc/mysql/en/Group_by_functions_and_modifiers.html)
- Joining tables, at <http://www.melonfire.com/community/columns/trog/article.php?id=148>
- Using subqueries, at <http://www.melonfire.com/community/columns/trog/article.php?id=204>